Arrays, cadenas y estructuras en C

GUÍA DE EXPLICACIONES PARA EL ALUMNADO
MF0964_3 · Desarrollo de elementos software para gestión de sistemas
12 de noviembre de 2026

Este apartado explica cómo agrupar datos relacionados. Primero utilizaremos arrays de números; después, cadenas de caracteres; y finalmente, estructuras que reúnen distintos tipos de información. Al unirlos podremos representar varios equipos informáticos de forma ordenada.

El material parte de variables, condicionales, bucles y funciones. Los ejemplos de explicación muestran cómo funciona cada concepto. Este documento contiene únicamente las explicaciones y sus ejemplos didácticos. El ejercicio de los cinco equipos y su solución completa no se incluyen aquí.

Tres formas de organizar información

Recurso	Qué agrupa	Ejemplo
Array	Varios elementos de un mismo tipo.	Cuatro notas enteras.
Cadena	Caracteres que forman texto y un terminador final.	Un nombre o un código.
Estructura	Campos que describen una entidad y pueden ser de tipos diferentes.	Código y cantidad de RAM de un equipo.

La idea que une todo el tema

Cada dato tiene un tipo, una ubicación dentro de la colección y una regla de uso. Conocer el número de elementos, respetar los índices y reservar espacio para el final de una cadena evita errores que C no detecta automáticamente durante la ejecución.

Recorrido de lectura: arrays e índices; bucles; arrays en funciones; cadenas; comparación de texto; estructuras; y combinación de estructuras con arrays.

1. Arrays: varios datos con un nombre

Un array es una colección de un número determinado de elementos del mismo tipo. En vez de declarar nota1, nota2, nota3 y nota4 por separado, utilizamos un nombre común y un índice para seleccionar cada nota.

```
int notas[4] = {4, 5, 7, 8};
```

Parte	Significado
int	El tipo de cada elemento es entero.
notas	Nombre del array.
[4] en la declaración	Se reservan cuatro elementos.
{4, 5, 7, 8}	Valores iniciales, en ese orden.

Un **índice** identifica una posición. En C los índices comienzan en cero. Si hay cuatro elementos, las posiciones válidas son 0, 1, 2 y 3.

Índice	Acceso al elemento	Valor inicial
0	notas[0]	4
1	notas[1]	5
2	notas[2]	7
3	notas[3]	8

Consultar y modificar

```
printf("%d\n", notas[0]);  // Consulta la primera nota.
notas[0] = 6;              // Modifica la primera nota.
```

Tras la asignación cambia solo el elemento seleccionado. El array continúa teniendo cuatro elementos. No estamos cambiando el índice ni añadiendo una nueva posición.

También podemos escribir int notas[] = {4, 5, 7, 8};. En esa declaración el compilador deduce el número de elementos a partir de los cuatro valores iniciales. No significa que el array crezca automáticamente.

2. Límites, tamaño e inicialización

Para un array de n elementos, los índices válidos van de 0 a n - 1. El número de elementos y el último índice no son lo mismo: un array de diez edades termina en el índice 9.

```
int edades[10];
// Primer elemento: edades[0]
// Ultimo elemento: edades[9]
// edades[10] queda fuera del array.
```

C no comprueba automáticamente todos los accesos. Leer o escribir fuera del array produce comportamiento indefinido: el lenguaje deja de garantizar qué ocurrirá. El programa puede parecer correcto, fallar o afectar a otros datos. Que no aparezca un error visible no demuestra que el acceso sea válido.

Revisar la condición de un recorrido

```
for (int i = 0; i < 4; i++)
{
    printf("%d\n", notas[i]);
}
```

Este fragmento supone que notas tiene cuatro elementos inicializados. Al empezar, i vale 0; el cuerpo usa los índices 0, 1, 2 y 3. Cuando i alcanza 4, la condición i < 4 es falsa y no se accede a notas[4]. Usar i <= 4 introduciría un acceso fuera del límite.

Tener espacio reservado no equivale a tener datos válidos

```
int valores[4] = {0};       // Los cuatro quedan a cero.
int otros[4] = {2, 3};      // Los restantes quedan a cero.
```

Un array local declarado como int datos[4];, sin inicialización, no queda automáticamente preparado con cuatro ceros. Hay que asignar cada elemento antes de leerlo. En estos ejemplos utilizamos inicializadores para conocer los valores de partida.

Una precaución adicional

Los elementos de un array también tienen un rango limitado por su tipo. Respetar el índice no evita, por sí solo, que una suma de enteros demasiado grande se desborde. Los ejemplos del tema utilizan valores pequeños y controlados.

3. Recorrer, acumular y contar

Este programa recorre cuatro notas, suma sus valores y cuenta cuántas alcanzan al menos 5. El contador del bucle selecciona la posición; suma y aprobados tienen responsabilidades diferentes.

```
#include <stdio.h>

int main(void)
{
    int notas[] = {4, 5, 7, 8};
    int suma = 0;
    int aprobados = 0;

    for (int i = 0; i < 4; i++)
    {
        suma += notas[i];
        if (notas[i] >= 5)
        {
            aprobados++;
        }
    }

    printf("Media: %.1f\n", suma / 4.0);
    printf("Aprobados: %d\n", aprobados);
    return 0;
}
```

i	notas[i]	suma al terminar la vuelta	aprobados
0	4	4	0
1	5	9	1
2	7	16	2
3	8	24	3

La suma final es 24. Al dividir entre 4.0 se obtiene 6.0, y hay 3 aprobados. El 5 está incluido por usar >= 5. La condición del for se comprueba una vez más con i = 4 y el cuerpo no se ejecuta.

suma += notas[i] equivale aquí a suma = suma + notas[i]. aprobados++ incrementa el contador en uno. La división por 4.0 conserva decimales; con operandos enteros, 25 / 4 produce 6, mientras que 25 / 4.0 produce 6.25.

4. Un array como parámetro de una función

En una declaración de parámetro como int notas[], la función no recibe automáticamente el número de elementos. Esa forma de parámetro se ajusta a un puntero al primer elemento. Por eso normalmente también se pasa la cantidad válida que se quiere recorrer.

```
#include <stdio.h>

void mostrar_notas(const int notas[], int cantidad)
{
    for (int i = 0; i < cantidad; i++)
    {
        printf("%d\n", notas[i]);
    }
}

int main(void)
{
    int notas[] = {4, 5, 7, 8};
    mostrar_notas(notas, 4);
    return 0;
}
```

La llamada mostrar_notas(notas, 4) proporciona el acceso a los datos y la cantidad que debe recorrer. La función usa cantidad para fijar su límite. El contrato exige que el array tenga, como mínimo, esa cantidad de elementos accesibles y que cantidad no sea negativa.

Qué aporta const

const int notas[] indica que la función no debe modificar los elementos a través de ese parámetro. Resulta apropiado para una función que solo muestra información. Sin esa restricción, una función podría modificar los elementos mediante el acceso al array recibido.

Cómo calcular una cantidad donde el array sí es visible

```
// Donde notas es el array real, no un parametro:
size_t cantidad = sizeof notas / sizeof notas[0];
```

sizeof notas obtiene el espacio total del array en bytes; sizeof notas[0], el espacio de un elemento. Su cociente da el número de elementos. size_t es el tipo de resultado de sizeof. Dentro de mostrar_notas no debemos usar esa fórmula para intentar recuperar el tamaño del array original: allí notas es un parámetro ajustado a puntero.

5. Cadenas: texto y terminador final

Una cadena en C es una secuencia de caracteres terminada por el carácter nulo '\0'. Habitualmente se almacena en un array de char con espacio suficiente para ese terminador.

```
char nombre[4] = "Ana";
```

Posición	Contenido	Papel
0	'A'	Primer carácter visible.
1	'n'	Segundo carácter visible.
2	'a'	Tercer carácter visible.
3	'\0'	Marca el final del texto.

El terminador no es el carácter '0'. No se imprime como una letra adicional: permite que las funciones de cadenas sepan dónde acaba el texto. Una cadena con tres letras ASCII necesita cuatro posiciones.

```
char nombre[] = "Ana";
printf("%s\n", nombre);    // Muestra Ana.
printf("%c\n", nombre[0]); // Muestra A.
```

Con [] y un inicializador, el compilador incluye el espacio para el terminador. %s muestra una cadena; %c muestra un carácter. Las comillas simples delimitan un carácter como 'A'; las dobles delimitan un literal de cadena como "Ana".

Capacidad del array y longitud del texto

char codigo[16] puede contener una cadena de hasta 15 bytes de texto y su terminador. Los códigos ASCII de este tema usan un byte por carácter. Con codificaciones como UTF-8, una letra acentuada puede ocupar más de un byte; por eso capacidad en bytes y cantidad de letras no siempre coinciden.

Una declaración como char nombre[3] = "Ana" puede dejar un array sin terminador. Aunque contenga esas tres letras, no es una cadena válida para usar con %s o strcmp. No confíes en que el compilador siempre lo rechazará.

6. Comparar y conservar cadenas válidas

Para comparar el contenido de cadenas utilizamos strcmp, cuya declaración está en string.h. Su resultado es cero cuando los textos son iguales; negativo o positivo cuando son distintos, según su orden de comparación.

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char nombre[] = "Ana";

    printf("Nombre: %s\n", nombre);
    printf("Inicial: %c\n", nombre[0]);

    if (strcmp(nombre, "Ana") == 0)
    {
        printf("El nombre coincide\n");
    }
    return 0;
}
```

La condición strcmp(nombre, "Ana") == 0 pregunta si ambos contenidos coinciden. Es una comparación sensible a mayúsculas: "Ana" y "ana" son textos distintos. No necesitamos memorizar un número negativo o positivo concreto para comprobar diferencia; basta con distinguir cero de no cero.

Por qué no usamos == para comparar texto

En expresiones habituales con arrays de caracteres, == compara los valores de dirección obtenidos al convertirlos en punteros, no recorre las letras. Para comparar el texto de cadenas válidas, utiliza strcmp.

Modificar texto exige comprobar el espacio

Puedes cambiar un carácter, por ejemplo nombre[0] = 'E';, si el índice es válido y el array es modificable. No puedes reasignar un array ya declarado con nombre = "Eva";. Para copiar un texto completo se necesita una operación de copia que respete la capacidad del destino y conserve el terminador.

Las funciones de cadenas no reciben siempre la capacidad del array. strcmp, por ejemplo, presupone cadenas válidas terminadas en '\0'. Si falta el final, puede leer fuera del espacio permitido. Antes de comparar o mostrar texto, asegúrate de que los datos cumplen esas condiciones.

7. Estructuras: campos de distintos tipos

Una estructura permite reunir datos que describen una misma entidad. Cada dato recibe el nombre de campo o miembro. A diferencia de un array de int, una estructura puede combinar campos de tipos diferentes.

```
struct Equipo
{
    char codigo[16];
    int ram_gib;
};
```

Esta declaración define el tipo struct Equipo. El código identifica el equipo y ram_gib representa, en este modelo sencillo, una cantidad entera de GiB de RAM. El punto y coma tras la llave final forma parte de la declaración.

```
struct Equipo equipo = {"PC01", 8};
printf("Codigo: %s\n", equipo.codigo);
printf("RAM: %d GiB\n", equipo.ram_gib);
equipo.ram_gib = 16;
```

El inicializador sigue el orden de los campos: primero codigo y después ram_gib. La variable equipo contiene ambos. El operador punto permite seleccionar un campo concreto, tanto para consultarlo como para modificarlo. El fragmento supone stdio.h incluido y las instrucciones de ejecución dentro de una función.

Expresión	Qué representa
equipo	La estructura completa que representa un equipo.
equipo.codigo	El array de caracteres del código.
equipo.codigo[0]	El primer carácter de ese código.
equipo.ram_gib	La cantidad de RAM guardada en ese campo.

Qué significa agrupar

La estructura expresa que ambos campos pertenecen al mismo equipo. No convierte el texto en un entero ni obliga a que los campos tengan el mismo tipo. Esta relación es parte del diseño de los datos.

8. Arrays de estructuras y datos desconocidos

Un array puede tener estructuras como elementos. Cada posición representa una entidad completa, con sus propios campos.

```
struct Equipo equipos[5];
// Declaracion: reserva cinco elementos.
// Inicializa sus campos antes de consultarlos.
```

Expresión	Significado
equipos[0]	Primer equipo completo.
equipos[0].codigo	Código del primer equipo.
equipos[0].ram_gib	RAM del primer equipo.
equipos[4]	Quinto y último equipo del array declarado.
equipos[0].codigo[0]	Primer carácter del código del primer equipo.

Para leer una expresión compuesta, ve de fuera hacia el dato seleccionado: equipos es la colección; [0] elige una estructura; .codigo elige su campo de texto; y el último [0] selecciona un carácter dentro de ese campo. Hay dos límites distintos: el del número de equipos y el de la capacidad del código.

Desconocido no significa cero

Si no se ha verificado la RAM de un equipo, no debemos inventar una cantidad ni confundir falta de información con cero. El programa necesita una regla explícita para representarlo. Dos diseños habituales son reservar un valor especial que no sea una medida válida o añadir un campo que indique si la medida está verificada.

La elección debe documentarse y aplicarse antes de comparar capacidades. Un valor especial negativo podría cumplir por accidente una condición como ram_gib < 8. Por tanto, primero hay que identificar si el dato es conocido y después realizar la comparación de capacidad.

Cuatro ideas para conservar

Un índice selecciona un elemento del array. El punto selecciona un campo de una estructura. Una cadena requiere terminador y espacio suficiente. Un dato desconocido requiere una representación y un tratamiento explícitos.